CSC 402 — MILESTONE 3-A

Database Design — Submission Template

Due: Week 12 • 5 marks • One submission per team (PDF)

How you will be graded (100 points = 5 marks)

Criterion	Section	Pts
ER diagram: all entities, keys and relationships with cardinality	Section 2	**30**
Relational schema: fields, types, keys and constraints	Section 3, 4	**30**
Screen-to-data map: every screen has its data	Section 5	**20**
Room plan: entities, DAOs and key queries	Section 6	**10**
GitHub evidence and AI Use Log	Section 7, 8	**10**

Cover Sheet

Team name / number	Team 05	**App name**	NEXA
GitHub repository link	https://github.com/Hebaasksr/Nexa	**Database type**	Other: Cloud Firestore (Firebase) + Room local cache

#	Student name	Student ID	Role in this milestone
1	Munirah Albuainain	2240007600	
2	Ghaida Hashem AlQarni	2240004359	Prepared the Relational Schema and Data Dictionary (Section 3) for five entities: User, Project, ProjectMember, Invitation, and Task.
3	Heba Khaled Askar	2240006064	Prepared the Firebase Plan (Section 6): Firestore collections, Security Rules, repositories, key queries and the Room cache. Connected Firebase Auth and Firestore in the app and uploaded the project and /docs/database to GitHub (Section 7).
4	Fatimah Emad Alarfaj	2240007580	
5	Jawaher Shaya Hazazi	2240006281	

1. Entities at a Glance

Entity (table name)	What it stores	Related to
User	Student profile	Project, ProjectMember, Task, TaskComment, ProjectFile, ChatMessage, Notification, Invitation

Entity (table name)	What it stores	Related to
Project	Project metadata and join code	User, ProjectMember, Task, ProjectFile, ChatMessage, Invitation, Notification
ProjectMember	User membership/role within projects	User, Project
Invitation	Pending invitations by email	Project, User
Task	Project tasks, status, deadline, assignee	Project, User, TaskComment
TaskComment	Discussion posted on a task	Task, User
ProjectFile	Metadata/URL for shared project files	Project, User, ChatMessage
ChatMessage	Messages in each project chat	Project, User, ProjectFile
Notification	User-facing reminder or update	User, Project

2. ER Diagram

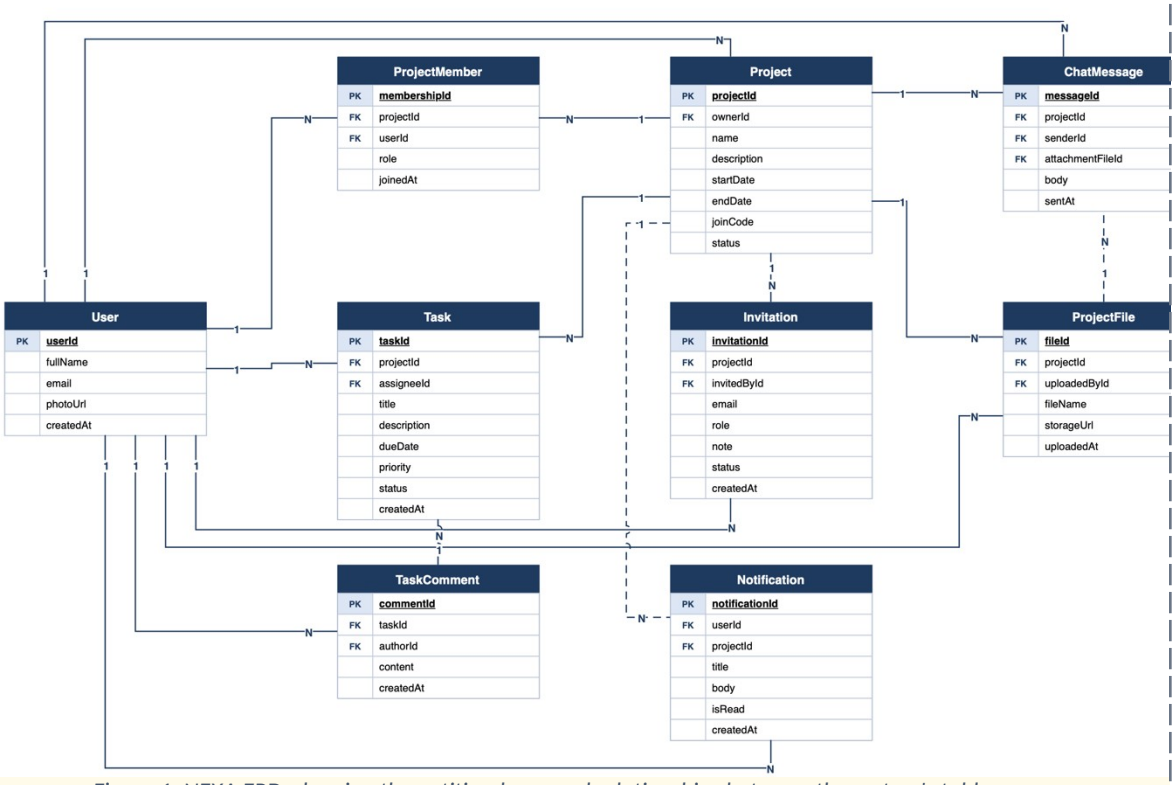


Figure 1: NEXA ERD, showing the entities, keys, and relationships between the system's tables.

3. Relational Schema & Data Dictionary

Table: User

Field	Type	Key	Constraints	Description
userId	String	PK	NOT NULL, UNIQUE	Unique user identifier
fullName	String		NOT NULL	User's full name
email	String		NOT NULL, UNIQUE	User's email address
photoUrl	String		Nullable	User's profile picture URL
createdAt	Timestamp		NOT NULL	Account creation timestamp

Table: Project

Field	Type	Key	Constraints	Description
projectId	String	PK	NOT NULL, UNIQUE	Unique project identifier
ownerId	String	FK → User	NOT NULL	User who created the project
name	String		NOT NULL	Project name
description	String		Nullable	Project description
startDate	Timestamp		NOT NULL	Project start date
endDate	Timestamp		NOT NULL	Project end date
joinCode	String		NOT NULL, UNIQUE	Code used to join the project
status	String		DEFAULT 'ACTIVE'	Project status

Table: ProjectMember

Field	Type	Key	Constraints	Description
membershipId	String	PK	NOT NULL, UNIQUE	Unique membership identifier
projectId	String	FK → Project	NOT NULL	Project the user belongs to
userId	String	FK → User	NOT NULL	User who joined the project
role	String		DEFAULT 'MEMBER'	User's role in the project
joinedAt	Timestamp		NOT NULL	Date and time the user joined

Table: Invitation

Field	Type	Key	Constraints	Description
invitationId	String	PK	NOT NULL, UNIQUE	Unique invitation identifier
projectId	String	FK → Project	NOT NULL	Project associated with the invitation
invitedById	String	FK → User	NOT NULL	User who sent the invitation
email	String		NOT NULL	Invited user's email address
role	String		DEFAULT 'MEMBER'	Assigned role for invited user
note	String		Nullable	Optional invitation message
status	String		DEFAULT 'PENDING'	Invitation status
createdAt	Timestamp		NOT NULL	Invitation creation timestamp

Table: Task

Field	Type	Key	Constraints	Description
taskId	String	PK	NOT NULL, UNIQUE	Unique task identifier
projectId	String	FK → Project	NOT NULL	Project containing the task
assigneeId	String	FK → User	Nullable	User assigned to the task
title	String		NOT NULL	Task title
description	String		Nullable	Task description
dueDate	Timestamp		Nullable	Task deadline
priority	String		DEFAULT 'MEDIUM'	Task priority level
status	String		DEFAULT 'TODO'	Current task status
createdAt	Timestamp		NOT NULL	Task creation timestamp

Table: TaskComment

Field	Type	Key	Constraints	Description
commentId	String	PK	NOT NULL, UNIQUE	Unique comment identifier
taskId	String	FK → Task	NOT NULL	Task the comment belongs to
authorId	String	FK → User	NOT NULL	User who wrote the comment
content	String		NOT NULL	Comment text
createdAt	Timestamp		NOT NULL	Comment creation timestamp

Table: ProjectFile

Field	Type	Key	Constraints	Description
fileId	String	PK	NOT NULL, UNIQUE	Unique file identifier
projectId	String	FK → Project	NOT NULL	Project the file belongs to
uploadedById	String	FK → User	NOT NULL	User who uploaded the file
fileName	String		NOT NULL	File name shown in the app
storageUrl	String		NOT NULL	Cloud Storage download URL
uploadedAt	Timestamp		NOT NULL	Upload date and time

Table: ChatMessage

Field	Type	Key	Constraints	Description
messageId	String	PK	NOT NULL, UNIQUE	Unique message identifier
projectId	String	FK → Project	NOT NULL	Project chat the message belongs to
senderId	String	FK → User	NOT NULL	User who sent the message
attachmentFileId	String	FK → ProjectFile	Nullable	File shared in the message, if any
body	String		Nullable, CHECK (body or attachmentFileId is not null)	Message text
sentAt	Timestamp		NOT NULL	Date and time the message was sent

Table: Notification

Field	Type	Key	Constraints	Description
notificationId	String	PK	NOT NULL, UNIQUE	Unique notification identifier
userId	String	FK → User	NOT NULL	User who receives the notification
projectId	String	FK → Project	Nullable	Related project, if any
title	String		NOT NULL	Notification title
body	String		NOT NULL	Notification message
isRead	Boolean		DEFAULT false	Whether the user has read it
createdAt	Timestamp		NOT NULL	Notification creation timestamp

4. Relationships

Relationship	Type	Implemented by (FK or junction table)	On delete
User owns many Projects	1:N	Project.ownerId → User.userId	RESTRICT

Relationship	Type	Implemented by (FK or junction table)	On delete
Users are members of many Projects	M:N	Junction table ProjectMember (userId → User, projectId → Project)	CASCADE
Project has many Invitations	1:N	Invitation.projectId → Project.projectId	CASCADE
User sends many Invitations	1:N	Invitation.invitedById → User.userId	CASCADE
Project has many Tasks	1:N	Task.projectId → Project.projectId	CASCADE
User is assigned many Tasks	1:N	Task.assigneeId → User.userId (nullable)	SET NULL
Task has many TaskComments	1:N	TaskComment.taskId → Task.taskId	CASCADE
User writes many TaskComments	1:N	TaskComment.authorId → User.userId	CASCADE
Project has many ProjectFiles	1:N	ProjectFile.projectId → Project.projectId	CASCADE
User uploads many ProjectFiles	1:N	ProjectFile.uploadedById → User.userId	RESTRICT
Project has many ChatMessages	1:N	ChatMessage.projectId → Project.projectId	CASCADE
User sends many ChatMessages	1:N	ChatMessage.senderId → User.userId	CASCADE
ProjectFile is attached to many ChatMessages	1:N	ChatMessage.attachmentFileId → ProjectFile.fileId (nullable)	SET NULL
User receives many Notifications	1:N	Notification.userId → User.userId	CASCADE
Project has many Notifications	1:N	Notification.projectId → Project.projectId (nullable)	CASCADE

Firestore has no foreign-key constraints, so the repositories apply these on-delete rules in batched writes.

5. Screen-to-Data Map

Screen	Reads (tables / fields)	Writes (insert / update / delete)
Create Account	—	Firebase Auth account; insert User (userId = Auth UID, fullName, email, createdAt)
Sign In	User (fullName, photoUrl) after Firebase Auth sign-in	—
Reset Password	— (Firebase Auth only)	— (Firebase Auth sends the reset email)
Dashboard (Home)	ProjectMember (userId) → Project (name, status, endDate); Task (status, dueDate) for counters and progress; Notification (isRead) for the bell	—

Screen	Reads (tables / fields)	Writes (insert / update / delete)
Projects	ProjectMember (userId) → Project (name, status) for All / Active / Archived	update Project.status (archive / unarchive)
New Project	—	insert Project (ownerId, name, description, startDate, endDate, joinCode); insert ProjectMember (role = OWNER); insert Invitation for each added member
Project Details (Overview, Tasks, Files, Team)	Project; Task (status for progress, dueDate for next milestone); ProjectMember + User (avatars); ProjectFile (Files tab)	—
My Tasks	Task (assigneeId = me; title, priority, status, dueDate); Project (name tag)	update Task.status (checkbox)
New Task (incl. validation error)	ProjectMember → Project (project list); ProjectMember + User (assignee list)	insert Task; insert Notification for the assignee
Task Created	Task (title, priority, dueDate); User (assignee fullName)	—
Task Details	Task; User (assignee); TaskComment + User (author)	update Task.status; insert TaskComment
Edit Task	Task; ProjectMember + User (assignee list)	update Task (title, description, assigneeId, dueDate, priority, status)
Delete Task (dialog)	Task (title)	delete Task and its TaskComments
Team	ProjectMember (role) + User (fullName, photoUrl); Invitation (status = PENDING)	—
Invite a Member	Project (name)	insert Invitation (email, role, note, status = PENDING)
Calendar	Task (dueDate in the selected month); Project (endDate)	—
Notifications	Notification (userId = me; title, body, isRead, createdAt); Invitation (email = my email, status = PENDING)	update Notification.isRead; accept invite: update Invitation.status + insert ProjectMember
Search	Project (name); Task (title); ProjectFile (fileName)	—
Project Chat	ChatMessage (projectId, body, sentAt) + User (sender); ProjectFile (attachment)	insert ChatMessage; insert ProjectFile when a file is attached
Files (Project Details tab, More → Files)	ProjectFile (fileName, uploadedById, uploadedAt)	insert ProjectFile (file uploaded to Cloud Storage)
More	User (fullName, photoUrl)	—
Profile & Settings	User (fullName, email, photoUrl)	update User (fullName, photoUrl); alert switches saved in DataStore

6. Firebase Plan

NEXA stores its data in Firebase, so this section adapts the Room Plan. Cloud Firestore is the main database (one collection per entity; document ID = primary key). Firebase Authentication manages accounts and Cloud Storage holds file content. As in our Milestone 2 architecture, Room caches Project and Task for offline use and DataStore keeps settings.

Firestore collections and Security Rules

Entity	Firestore path (document ID)	Security Rules (who can read / write)
User	users/{userId} (userId = Firebase Auth UID)	Read: signed-in users. Write: only the user (request.auth.uid == userId)
Project	projects/{projectId}	Read: project members. Create: any signed-in user as owner. Update / delete: owner
ProjectMember	projectMembers/{projectId_userId}	Read: members of the project. Write: project owner, or the user joining by code or invitation
Invitation	invitations/{invitationId}	Read: sender, project owner and the invited email. Create: members. Update status: invited user
Task	tasks/{taskId}	Read / write: members of the task's project
TaskComment	taskComments/{commentId}	Read: project members. Create: members (authorId == uid). Delete: author
ProjectFile	projectFiles/{fileId}; file content in Storage at projects/{projectId}/files/{fileId}	Read / upload: project members. Delete: uploader or owner
ChatMessage	chatMessages/{messageId}	Read: project members. Create: members (senderId == uid). No edits
Notification	notifications/{notificationId}	Read / update isRead: only the user (userId == uid)

Repositories (Firestore) and DAOs (Room cache)

Repository / DAO	Key queries (name + purpose)	Used by (ViewModel / use case)
AuthRepository (Firebase Auth + users)	signUp(fullName, email, password): Auth account + users/{userId} signIn(email, password); sendPasswordReset(email) getUser(userId): Flow<User>; updateProfile(fullName, photoUrl)	AuthViewModel (LoginUser)
ProjectRepository (projects, projectMembers, invitations)	getMyProjects(userId): Flow<List<Project>> via ProjectMember createProject(project): Project + ProjectMember (OWNER) in one batch joinByCode(joinCode): find Project, add ProjectMember archiveProject(projectId): status = ARCHIVED getMembers(projectId); sendInvitation(invitation); acceptInvitation(invitationId)	ProjectViewModel (CreateProject, JoinProject, CalculateProgress)

Repository / DAO	Key queries (name + purpose)	Used by (ViewModel / use case)
TaskRepository (tasks, taskComments)	getTasksByProject(projectId): Flow<List<Task>> ordered by dueDate getMyTasks(userId, status?): tasks where assigneeId == userId getTasksDueBetween(start, end): Calendar and upcoming deadlines createTask(task); updateTask(task); setStatus(taskId, status); deleteTask(taskId) getComments(taskId): Flow<List<TaskComment>>; addComment(comment)	TaskViewModel, CalendarViewModel (CreateTask, AssignTask, CompleteTask, GetUpcomingDeadlines)
FileRepository (projectFiles + Cloud Storage)	getFiles(projectId): Flow<List<ProjectFile>> newest first uploadFile(projectId, uri): upload to Storage, then save fileName and storageUrl deleteFile(fileId)	FilesViewModel
ChatRepository (chatMessages)	listenMessages(projectId): real-time Flow ordered by sentAt sendMessage(message)	ChatViewModel
NotificationRepository (notifications)	getNotifications(userId): Flow ordered by createdAt (newest first) markRead(notificationId); markAllRead(userId)	NotificationViewModel
ProjectDao (Room cache)	getAll(): Flow<List<ProjectEntity>> upsertAll(projects); deleteAll(): refreshed from Firestore	ProjectRepository (offline Dashboard and Projects)
TaskDao (Room cache)	getByProject(projectId): Flow<List<TaskEntity>> getDueBetween(start, end); upsertAll(tasks); deleteById(taskId)	TaskRepository (offline My Tasks and Calendar)

Entity classes (Kotlin names)	Firestore models (data classes, @DocumentId on the primary key): User, Project, ProjectMember, Invitation, Task, TaskComment, ProjectFile, ChatMessage, Notification Room cache: ProjectEntity, TaskEntity (@Entity) in NexaDatabase (@Database)
Type converters / enums needed	Enums saved as String: ProjectStatus (ACTIVE, ARCHIVED), MemberRole (OWNER, MEMBER), InvitationStatus (PENDING, ACCEPTED, DECLINED), TaskStatus (TODO, IN_PROGRESS, DONE), TaskPriority (LOW, MEDIUM, HIGH) Firestore maps Timestamp automatically (@ServerTimestamp on createdAt, sentAt, uploadedAt); Room needs a Converters class for Timestamp ↔ Long (epoch millis)
Sample (seed) data: what and where from	3 projects, 5 tasks, 4 notifications and 5 files from SAMPLE_PROJECTS, SAMPLE_TASKS, SAMPLE_NOTIFICATIONS and DEMO_FILES in model/Models.kt, written to Firestore once by a debug-only seedDemoData() for the signed-in user

Remote source / mock API (if any)	Firebase project nexa-8fbb1 (app/google-services.json): Firebase Authentication (email and password), Cloud Firestore (9 collections above), Cloud Storage for Firebase (file content), Firebase Cloud Messaging for deadline reminders. No mock API.

7. GitHub Evidence

☒ Files committed to /docs/database: ER diagram image, schema (this PDF or a .md file)
☐ Commit tagged milestone-3-stage-1
☐ All Milestone 3-A issues closed; each member has at least one contribution
☒ Repository is accessible to the instructor

8. AI Use Log

Tool	Where used	What it was used for	What we changed or checked ourselves
Claude (Anthropic)	Sections 3 (TaskComment, ProjectFile, ChatMessage, Notification), 4, 5, 6 and 7; GitHub upload	Drafted these sections from our ER diagram, Milestone 2 screens and app code; adapted the Room Plan to Firebase; prepared /docs/database	Heba checked the Firebase Plan against our Firebase project and app code; the team checked every name against the ER diagram

☐ We can explain every table, key and relationship in this design without help.
☐ The design matches our Milestone 2 architecture and navigation.
☐ Table and field names are the same in the diagram, schema and Room plan.